

Video Transcript

Video 2: Change Data Capture (CDC) Overview (04:59)

Change is inescapable. It is one of the constants in life. In fact, we as humans constantly track that change. There's even an expression right fear of missing out. Now, when it comes to data, the same thing is true. Once you create a database, there are a number of parties that are interested in changes to that database. Now, when it comes to data capture, as you might imagine this is not new. It has been around for decades and perhaps one of the most tangible examples is that of vanity phone numbers. Most countries around the world have pretty numbers that they often sell to businesses that then map to regular numbers. Now, every single time there is a change to one of these, the hundreds of database servers around the phone network need to be updated to reflect that change. Something similar happens with the domain name server system, the mapping of pretty domain names like mit.edu to IP numbers around the world. Every single time there's a change to one of those, that update needs to happen, in this case globally.

Now, one of the twists when it comes to change data capture is that we're looking at real time data or as close as we can get to it. You might be wondering, well, what's the big deal? We have a database and we have an application. Why can't we do that? Well, as it turns out, when that application scales, you get more needs. You might no longer be able to meet the demand and you need to add additional performance. Which means you will add a cache. Same thing happens to being able to find information within your application. You may need a search index and an additional one would be a data warehouse. these needs grow and you ultimately end up with a system of record, a source of truth, and then the right data. Once you do that, you need to synchronize that information. And when it comes to the ways in which you can do that, there are a number of them. I'll list here four of them, dual rights, looking at it from the network, looking at it from the database, and then periodic queries.

Now, let's look at each one of these in turn. As you can see here, this is dual writes, and this might be your first reaction. Let's just update our source code and let's write to the number of places that need this data. Well, it turns out that's a pretty bad idea. For one, you have to

update your source code. This might not be something you want to do. The other is that this turns out to be non-transferable. That is the code that you're writing is specific to that application. And in general, there are a number of problems that emerge because once again, this might be a natural thing to do at first that simply cause all sorts of problems. And data engineers like to say friends don't let friends do dual writes.

Now, another one that you might think of is network level sniffing. This is similar to the screen scraping that Google does. They navigate the World Wide Web and scrape the pages and try to create a data model from it. Now, this is complex enough on its own. However, you can imagine that one of the additional things that you miss if you take this approach are the intermediate changes of state that happen inside the database. The other issue as well is that if this application crashes, the sniffing application crashes. You miss everything that happens because there is no redundancy to it. Now, the third way you might look at this is to look at it from the database perspective. And if you are a database person, you tend to think in terms of databases. As you can see here, there are a number of issues with addressing it that way. As mentioned, a lot of these issues are emerged as your application scales. And if you're adding additional load, you know you might have to do multiple writes to multiple sources. This is going to slow down further your application.

Now, you can see additional scenarios there where you know if a transaction fails, you'll have less updates. But in general, we have a lot of the same issues when it comes to transferability that we had in dual writes. The logic that you're writing is specific to this application. Now, a fourth way, and this is a pretty good way when it comes to DIY type of approaches, is to do periodic queries and leverage timestamps in the database. If done right, this can be pretty low overhead. Now, it does have some disadvantages that you can see there. Depending on how you write it, you could address those or they might be big deals. Now, the last thing to mention when it comes to CDC is that you're looking really at three core components. You're looking at the detection of the change, you're looking at capturing that change, and you're looking at propagating the change. When you think in terms of CDC, think in terms of these three core components.